

CSE3010 - COMPUTER VISION

IntrudeAware

Intelligent Video Surveillance and Scene Analysis

Adaptive YOLO Detection, Multi-Object Tracking, Motion Analysis and
Restricted-Zone Events

Student Name:	Shlok Khanna
Registration No.:	24BAI10235
Program:	B.Tech. CSE-AIML
Course:	CSE3010 Computer Vision
Project Type:	VITyarthi - Build Your Own Project
Academic Year:	2026

Final Report

Prepared for VITyarthi project guidelines.

Abstract

IntrudeAware is a modular computer-vision application that transforms uploaded video into semantic detections, persistent object tracks, motion measurements, restricted-zone state transitions, and visual analytics. The system combines image preprocessing, Ultralytics YOLO semantic detection, a persistent centroid tracker, sparse Lucas-Kanade optical flow, rule-based zone-event detection, and a Streamlit dashboard. The design keeps the tracker and event engine independent of the detector, allowing the semantic detector to be refreshed less frequently than the video frame rate. An adaptive scheduling policy uses smoothed object velocity and restricted-zone activity to request more frequent YOLO inference during meaningful motion while reducing unnecessary inference in calmer scenes. A short refresh cooldown is added to reduce scheduler oscillation caused by noisy frame-to-frame motion.

The project is intentionally modular and explainable. It includes automated tests, configurable CPU/CUDA inference, browser-compatible video output, timestamped ENTRY/EXIT events, CSV export, design diagrams, and an evaluation plan. The implementation aligns with the course emphasis on image preprocessing, motion analysis, object detection/tracking, and development of computer-vision applications. The final report separates measured project results from external vendor benchmarks and includes a reproducible evaluation protocol rather than inventing unmeasured performance values.

Keywords: computer vision, YOLO, object detection, object tracking, optical flow, restricted-zone detection, adaptive inference, Streamlit.

Contents

Abstract	i
1 Introduction	1
1.1 Project Motivation	1
1.2 Project Goals	1
2 Problem Statement, Scope and Objectives	3
2.1 Problem Statement	3
2.2 Scope	3
2.3 Objectives	4
3 Requirements Analysis	5
3.1 Functional Requirements	5
3.2 Non-Functional Requirements	5
3.3 Target Users	6
4 System Architecture and Design	7
4.1 High-Level Architecture	7
4.2 Process Workflow	7
4.3 Use Case Diagram	8
4.4 Sequence Diagram	9
4.5 Class and Component View	9
4.6 Design Decisions	9
5 Computer Vision Methodology	11
5.1 Image Preprocessing	11
5.2 Semantic Object Detection with YOLO	11
5.2.1 Model-size rationale	11

5.3	Object Tracking	12
5.4	Optical Flow	12
5.5	Restricted-Zone Event Detection	12
5.6	Adaptive YOLO Frame Scheduling	12
6	Software Implementation	13
6.1	Technology Stack	13
6.2	Repository Structure	13
6.3	Important Modules	13
6.4	Representative Scheduling Code	14
7	User Interface and Results Visualization	15
7.1	Dashboard	15
7.2	Video Overlay	15
7.3	Event Dashboard	16
7.4	Final Runtime Configuration and Runtime Evidence	16
7.5	Observed Event-Log Evidence	19
8	Testing and Evaluation	21
8.1	Automated Testing	21
8.2	Evaluation Methodology	21
8.3	Performance Metrics	21
8.4	Benchmark Status	22
9	Discussion	23
9.1	Strengths of the Design	23
9.2	Limitations	23
9.3	Ethical and Privacy Considerations	24
10	Challenges Faced and How They Were Addressed	25
11	Future Enhancements	26
12	VITyarthi Requirement Traceability	27
13	Conclusion	28
A	Installation and Usage	29

A.1	Windows PowerShell	29
A.2	Recommended first-run settings	29
B	Selected Source Files	30
B.1	Core dependencies	30
B.2	Project Configuration	30
C	References	31

List of Figures

4.1	IntrudeAware system architecture.	7
4.2	IntrudeAware processing workflow.	7
4.3	Use case diagram.	8
4.4	Sequence-oriented interaction diagram for one video-processing session. .	9
4.5	Class/component relationships used by the project.	9
7.1	IntrudeAware dashboard at runtime with upload, motion, YOLO, and restricted-zone controls.	15
7.2	Illustrative IntrudeAware output overlay generated from the project drawing functions.	16
7.3	IntrudeAware dashboard showing the final project settings before processing, including YOLO26n, CPU inference, frame skipping, and adaptive scheduling. .	17
7.4	IntrudeAware processed-video view showing semantic labels, persistent track IDs, restricted-zone highlighting, and in-zone status.	17
7.5	IntrudeAware processing summary and restricted-zone event dashboard. .	18
7.6	IntrudeAware event-log tail, latest event, processing metadata, and saved output path.	18
7.7	Observed ENTRY and EXIT event counts in the supplied runtime CSV log. .	19

List of Tables

7.1	Visible runtime settings recorded in the supplied screenshots.	19
7.2	Summary of the supplied runtime event CSV.	20
7.3	First twelve records from the supplied runtime event log.	20
12.1	Traceability of submission requirements.	27

Chapter 1

Introduction

The supplied CSE3010 syllabus states that the course covers the theoretical and practical aspects of computing with images, image formation and analysis, major computer-vision methods, and the development of computer-vision applications [1]. Its indicative practical work includes image preprocessing and edge detection, segmentation, optical flow, object detection and tracking from video, dynamic-background surveillance, and related 3D-vision tasks. IntrudeAware was designed to combine several of these concepts into one coherent application rather than implementing isolated laboratory exercises.

The VITyarthi guide requires an original project aligned with the subject, at least three major functional modules, clear input/output, a logical workflow, non-functional requirements, modular implementation, testing, design artifacts, and a final report [2]. IntrudeAware therefore treats the video-analysis pipeline, event engine, dashboard, and adaptive inference scheduler as one integrated system.

1.1 Project Motivation

Continuous video contains a large number of frames, while the semantic content of the scene often changes more slowly than the raw frame rate. Running a semantic detector on every frame can therefore be computationally expensive. At the same time, zone-entry events and object motion require timely updates. The project addresses this tension by separating semantic detection from the downstream tracking and event layers.

1.2 Project Goals

1. Build a usable computer-vision application for uploaded video.
2. Demonstrate CSE3010 concepts through an integrated pipeline.
3. Preserve persistent track IDs between semantic detector refreshes.
4. Produce interpretable restricted-zone ENTRY/EXIT events with timestamps.
5. Reduce unnecessary YOLO inference with adaptive scheduling while retaining frame-level visualization and event updates.

6. Provide a reproducible evaluation and documentation package.

Chapter 2

Problem Statement, Scope and Objectives

2.1 Problem Statement

Manual monitoring of continuous video is repetitive and makes it difficult to consistently record object movement and zone events. IntrudeAware processes an uploaded video, detects supported semantic object classes, assigns persistent track IDs, estimates motion, monitors a user-defined restricted region, and presents the results as an annotated video and event dashboard.

2.2 Scope

The final project focuses on uploaded video files. The system performs:

- frame preprocessing and enhancement;
- YOLO-based semantic detection;
- multi-object tracking with persistent IDs and intermediate-frame prediction;
- sparse optical-flow analysis;
- restricted-zone ENTRY/EXIT event detection;
- adaptive YOLO frame scheduling with velocity smoothing and a cooldown;
- interactive visualization and CSV event export.

The project is an academic prototype and is not presented as a safety-critical surveillance product.

2.3 Objectives

1. Implement an end-to-end computer-vision video pipeline.
2. Demonstrate preprocessing, filtering, edge detection, motion computation, object detection, tracking, and event analysis.
3. Maintain stable object identities across detector refreshes.
4. Generate non-duplicated zone entry events and explicit exit events.
5. Compare inference scheduling strategies using reproducible measurements.
6. Deliver modular source code and formal design documentation.

Chapter 3

Requirements Analysis

3.1 Functional Requirements

1. The system shall accept MP4, AVI, MOV, and MKV video files.
2. The system shall preprocess video frames using resizing, Gaussian filtering, contrast enhancement, and Canny edge detection.
3. The system shall run YOLO semantic detection on supported classes and expose model size, confidence, and device controls.
4. The system shall maintain persistent track IDs across detection refreshes.
5. The system shall continue to update track positions on intermediate frames using the existing tracker prediction logic.
6. The system shall monitor a rectangular restricted zone.
7. The system shall generate ENTRY and EXIT events when a track changes zone state.
8. The system shall retain zone state during short detection gaps to avoid duplicate entries.
9. The system shall show track IDs, semantic labels, and IN ZONE status on the output frame.
10. The system shall show dashboard metrics and provide event CSV export.

3.2 Non-Functional Requirements

Requirement	Description
Performance	Adaptive scheduling should reduce unnecessary YOLO calls while preserving frame-level tracker, event, and overlay updates.

Usability		All major settings shall be available from the Streamlit sidebar.
Reliability		Invalid inputs and detector failures should not silently corrupt event state.
Maintainability		Detector, tracker, motion, event, visualization, and UI components are separated into modules.
Scalability		The semantic detector can be changed without redesigning the event and visualization layers.
Logging		Processing and event information can be inspected through the dashboard and CSV export.
Resource efficiency	effi-	Model scale, device, frame skipping, smoothing, and cooldown are configurable.

3.3 Target Users

The primary users are computer-vision students, academic demonstrators, researchers prototyping video-analysis pipelines, and developers learning applied computer vision.

Chapter 4

System Architecture and Design

4.1 High-Level Architecture

Figure 4.1 shows the separation between video input, preprocessing, semantic detection, tracking, motion analysis, event analysis, visualization, and reporting.

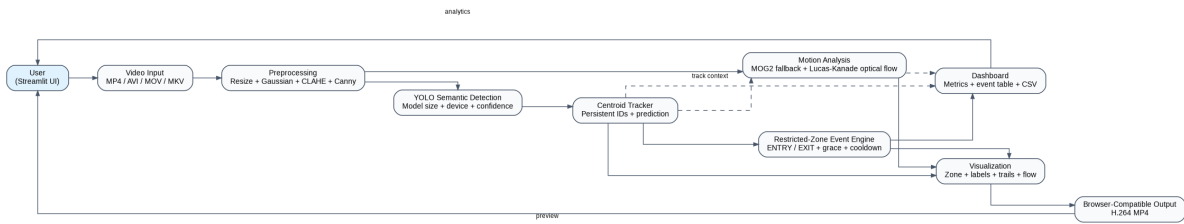


Figure 4.1: IntrudeAware system architecture.

4.2 Process Workflow

The workflow is frame-oriented. YOLO may refresh only when the adaptive scheduler requests it. Intermediate frames are still passed through tracker prediction, optical-flow computation, zone state updates, overlay generation, and output writing.



Figure 4.2: IntrudeAware processing workflow.

4.3 Use Case Diagram

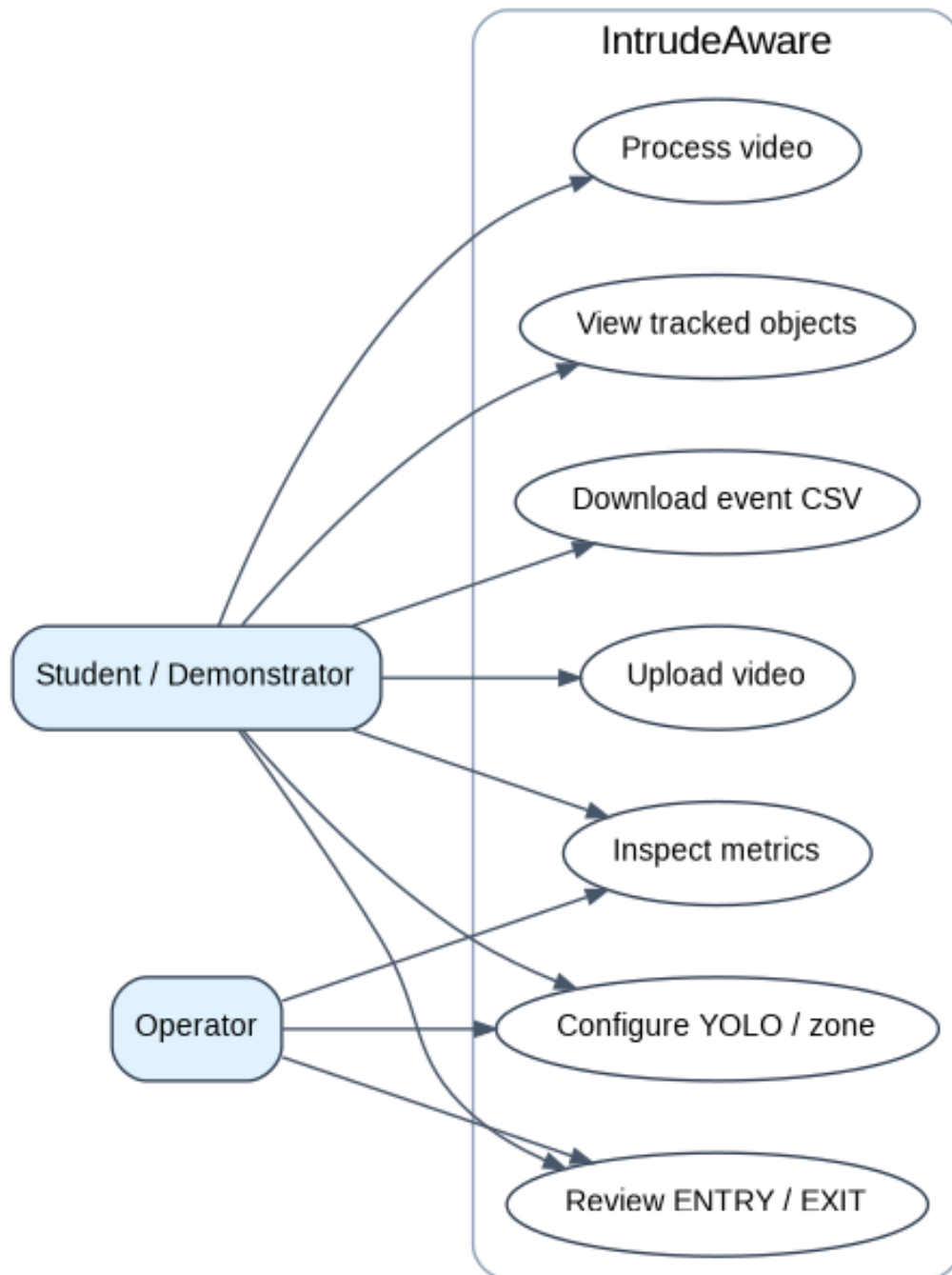


Figure 4.3: Use case diagram.

4.4 Sequence Diagram

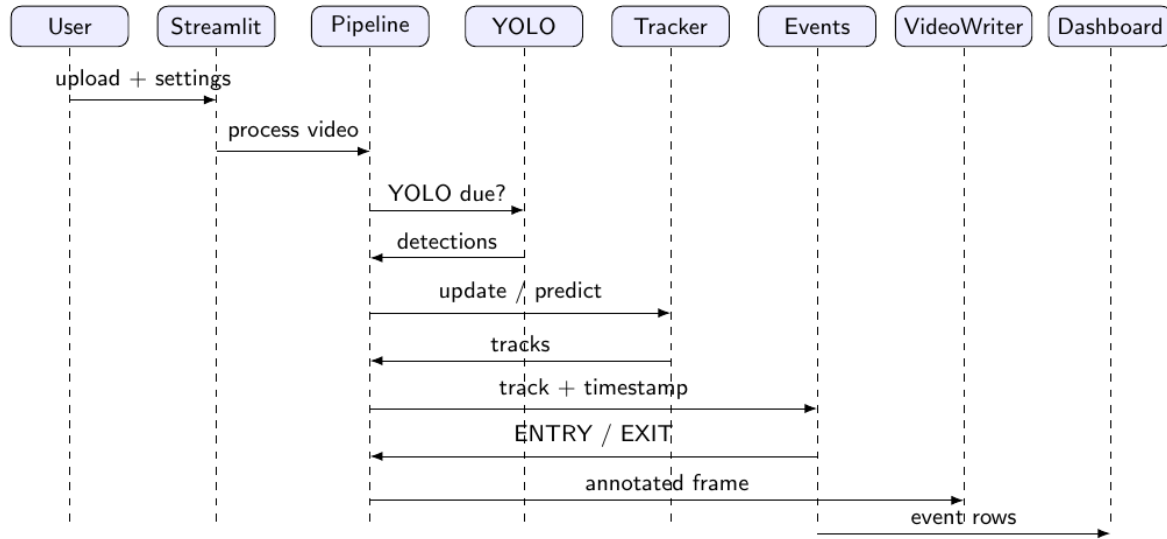


Figure 4.4: Sequence-oriented interaction diagram for one video-processing session.

4.5 Class and Component View

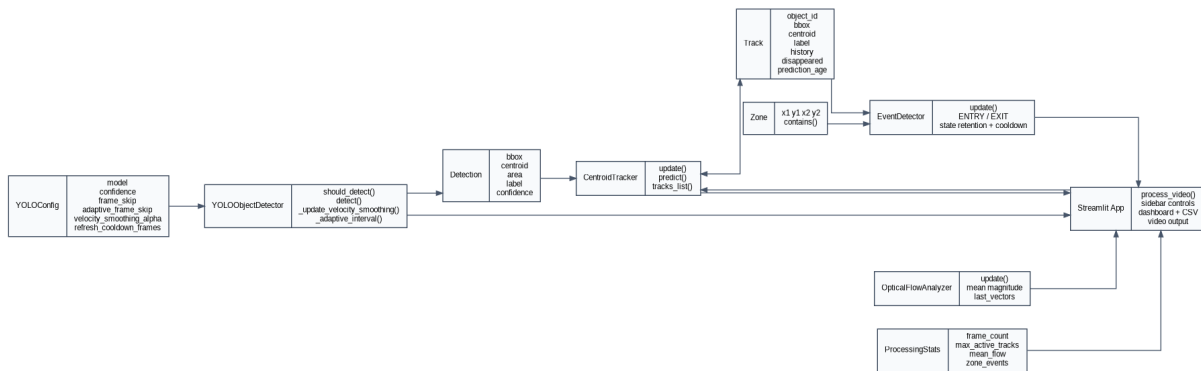


Figure 4.5: Class/component relationships used by the project.

4.6 Design Decisions

- **Detector abstraction:** YOLO detections are converted into the existing **Detection** structure, so the tracker and event engine do not depend on Ultralytics internals.
- **Persistent tracking:** the existing centroid tracker remains responsible for track IDs, not the semantic detector.
- **Event state retention:** zone state is retained through short missing periods to prevent duplicate ENTRY events caused by temporary detector loss.

- **Adaptive scheduling:** semantic inference frequency is controlled separately from the output frame rate.
- **Configuration:** model, device, confidence, frame skip, smoothing, cooldown, and zone geometry are exposed as parameters.

Chapter 5

Computer Vision Methodology

5.1 Image Preprocessing

Each frame is resized to a working width, converted for processing, smoothed with a Gaussian kernel, enhanced with CLAHE, and optionally converted to an edge map using Canny. These operations correspond to the low-level image-processing concepts emphasized in the course syllabus [1].

5.2 Semantic Object Detection with YOLO

IntrudeAware uses a pretrained Ultralytics YOLO26 detector. The application currently exposes the nano, small, medium, large, and extra-large detection scales and passes the selected device explicitly to the detector. Ultralytics documents the YOLO26 family, the five detection scales, and Python inference through the `YOLO()` interface [5].

The project maps the model’s COCO class names to a small application vocabulary:

YOLO class	IntrudeAware label
person	PERSON
bicycle	BIKE
motorcycle	BIKE
car	CAR
bus	BUS
truck	TRUCK

Unsupported classes are ignored by the application layer.

5.2.1 Model-size rationale

The project defaults to YOLO26n because the smallest model provides a practical baseline for an academic laptop while preserving the same API shape used by larger variants. Ultralytics publishes model-family benchmark figures for its own reference environments;

these values are not treated as IntrudeAware performance measurements [5]. The final project evaluation must use the actual target machine and fixed evaluation video.

5.3 Object Tracking

The tracker associates detections to existing tracks using nearest-centroid matching subject to a configurable maximum distance. Every track stores an object ID, bounding box, centroid, label, trajectory history, disappearance count, and prediction age.

When YOLO is skipped on an intermediate frame, the tracker estimates the next centroid from the last two trajectory points. This preserves visualization continuity and supplies a predicted state to the event detector.

5.4 Optical Flow

Sparse Lucas-Kanade optical flow estimates local point motion between consecutive preprocessed frames. The system records flow vectors and computes a mean magnitude used in the dashboard and as contextual motion evidence.

5.5 Restricted-Zone Event Detection

A zone is represented by a rectangular region in frame coordinates. A track generates an ENTRY when it transitions from outside to inside and an EXIT when it transitions from inside to outside. Two safeguards reduce duplicate events: a missing-frame grace period and a re-entry cooldown.

5.6 Adaptive YOLO Frame Scheduling

Let v_t be the maximum observed track speed in pixels per frame at time t . The scheduler maintains an exponentially weighted moving average:

$$\hat{v}_t = \alpha v_t + (1 - \alpha)\hat{v}_{t-1}, \quad 0 < \alpha \leq 1.$$

The desired detector interval is then selected from three levels: the configured maximum interval for calm motion, a shorter interval for moderate motion, and one-frame refresh for fast motion or an active in-zone track. A minimum refresh cooldown is applied after the desired interval is selected.

This separates semantic inference frequency from the output frame rate: all frames continue through tracking, event analysis, visualization, and video writing even when YOLO is not invoked.

Chapter 6

Software Implementation

6.1 Technology Stack

Layer	Technology
Language	Python
Image processing	OpenCV
Numerical operations	NumPy
Semantic detection	Ultralytics YOLO26
Data handling	Pandas
Dashboard	Streamlit
Video encoding helper	imageio-ffmpeg
Testing	Pytest
Version control	Git / GitHub

6.2 Repository Structure

The repository is divided by responsibility so that changes to the semantic detector do not force changes to the tracker, event logic, or dashboard. The source tree includes the preprocessing, detection, tracking, motion, analysis, visualization, utility, configuration, evaluation, and test modules.

6.3 Important Modules

Module	Role
<code>app.py</code>	Streamlit interface and end-to-end video processing loop.
<code>detection/yolo_detector.py</code>	YOLO loading, semantic filtering, adaptive scheduling, velocity smoothing, and cooldown.

tracking/tracker.py	Persistent centroid tracks and intermediate-frame prediction.
analysis/event_detection.py	Zone state machine and timestamped ENTRY/EXIT events.
motion/optical_flow.py	Sparse Lucas-Kanade motion analysis.
visualization/drawing.py	Restricted-zone overlay, labels, trails, and frame metrics.
analysis/statistics.py	Processing statistics.

6.4 Representative Scheduling Code

Listing 6.1: Adaptive scheduling rule used by YOLOObjectDetector

```
smoothed_speed = self._update_velocity_smoothing(tracks)

any_inside = any(
    track.disappeared == 0 and _inside_zone(track, zone_bounds)
    for track in tracks
)

if any_inside:
    return 1
if smoothed_speed >= self.cfg.high_speed_px_per_frame:
    return 1
if smoothed_speed >= self.cfg.medium_speed_px_per_frame:
    return min(2, self.cfg.frame_skip)
return self.cfg.frame_skip
```

Chapter 7

User Interface and Results Visualization

7.1 Dashboard

The Streamlit dashboard provides controls for model size, device, confidence threshold, maximum frame skip, adaptive scheduling, velocity smoothing, refresh cooldown, optical-flow vectors, Canny overlay, and restricted-zone geometry.

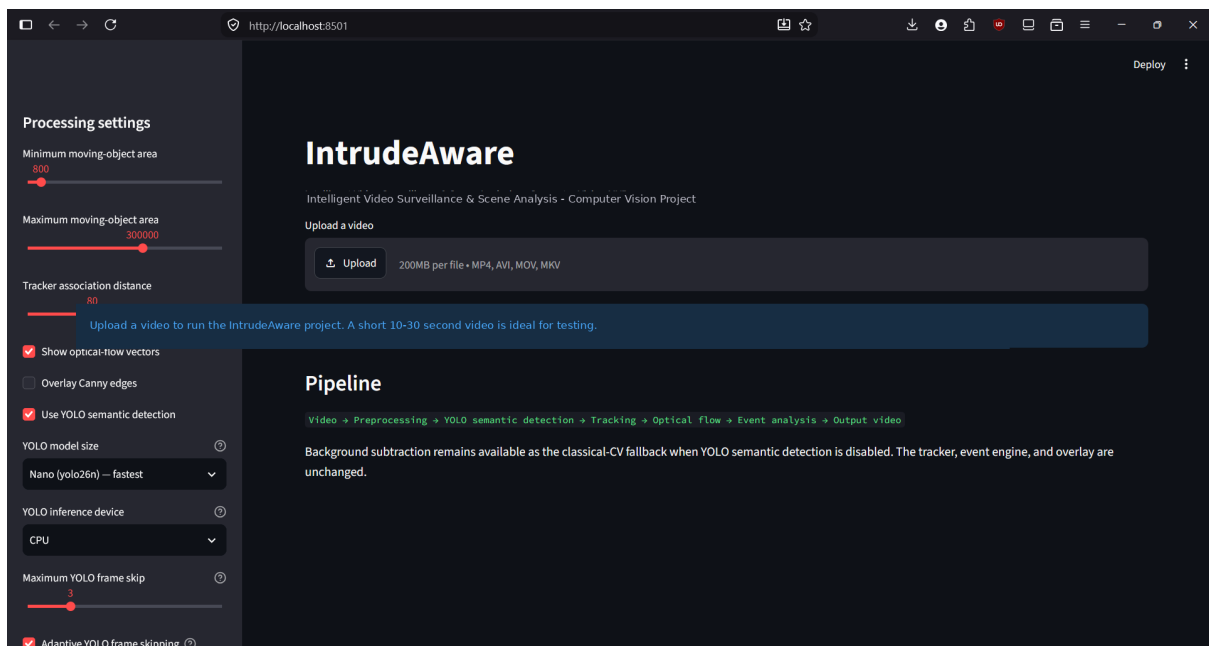


Figure 7.1: IntrudeAware dashboard at runtime with upload, motion, YOLO, and restricted-zone controls.

7.2 Video Overlay

The processed video highlights the restricted zone, labels every active track with an ID and semantic class, and appends IN_ZONE for tracks currently inside the protected region.

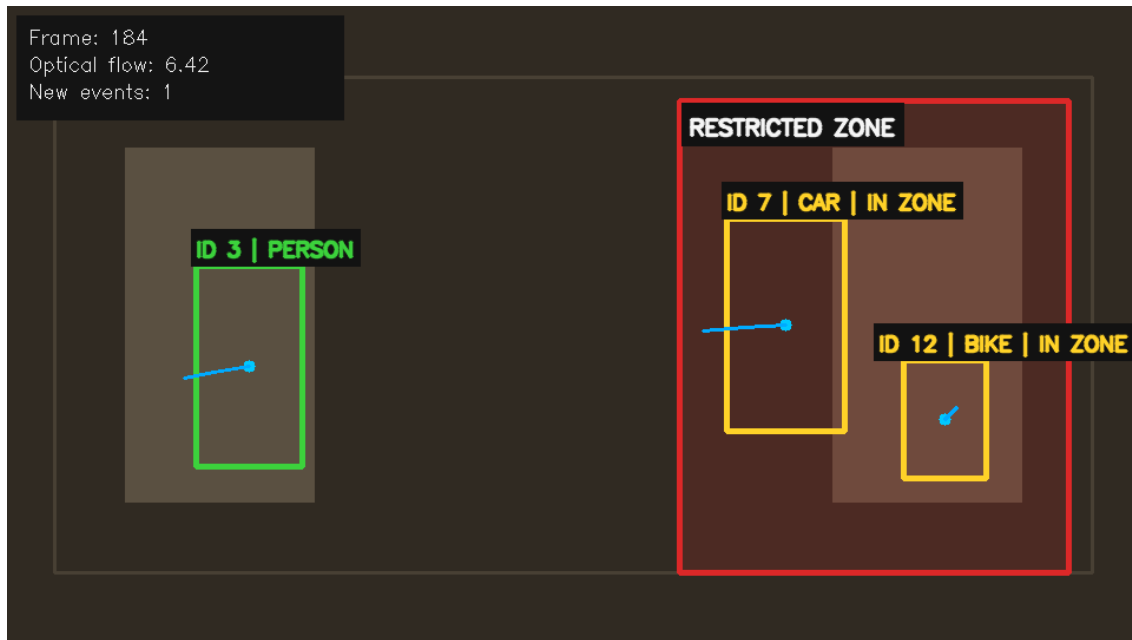


Figure 7.2: Illustrative IntrudeAware output overlay generated from the project drawing functions.

7.3 Event Dashboard

Each event row contains the relative video timestamp, frame number, track ID, ENTRY/EXIT status, and a human-readable message. The table can be filtered by event status and downloaded as CSV. The relative timestamp is computed as frame index divided by the video FPS, so it remains tied to the analyzed video rather than the system clock.

7.4 Final Runtime Configuration and Runtime Evidence

The final settings view supplied with the project records the processing thresholds and the YOLO/adaptive-inference configuration used for the run.

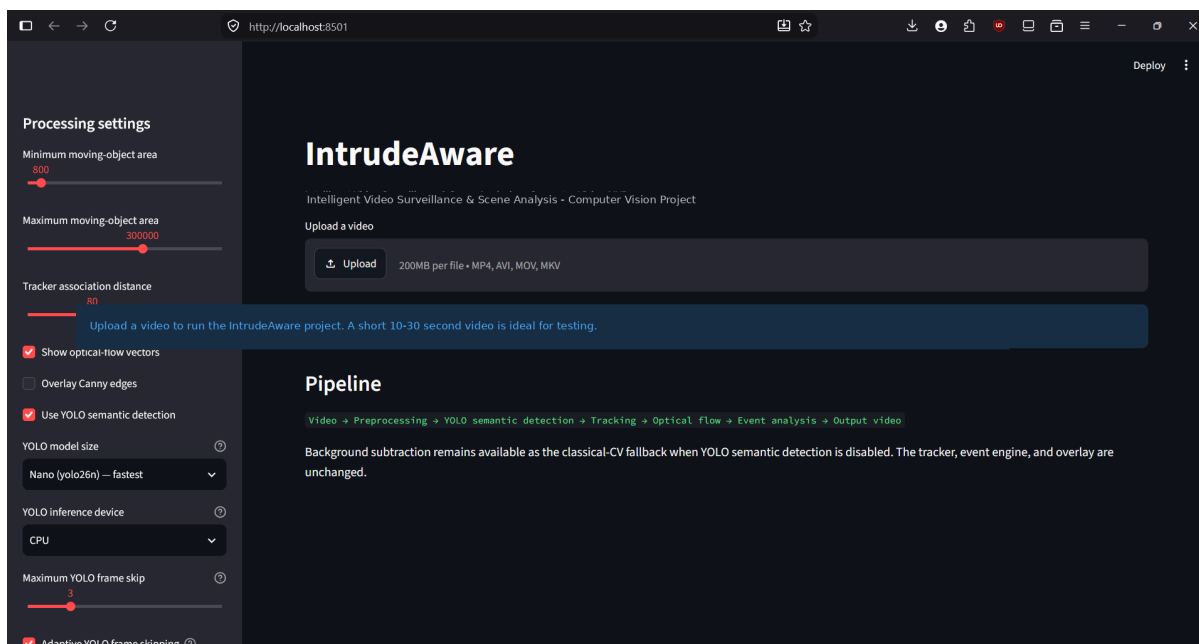


Figure 7.3: IntrudeAware dashboard showing the final project settings before processing, including YOLO26n, CPU inference, frame skipping, and adaptive scheduling.

The supplied runtime screenshots document the final IntrudeAware configuration and its observable outputs. The processed-video view shows persistent track IDs, semantic labels, the highlighted restricted zone, optical-flow information, and in-zone status. The event-dashboard screenshots show timestamped ENTRY/EXIT rows, track IDs, event messages, and the processing summary.

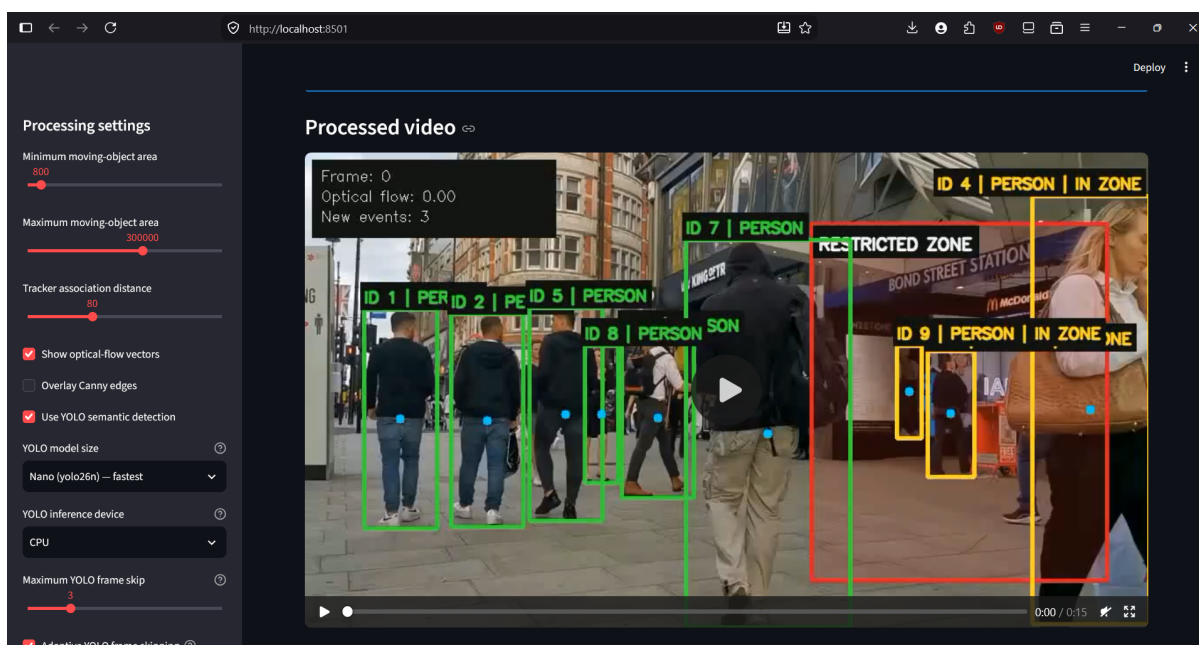


Figure 7.4: IntrudeAware processed-video view showing semantic labels, persistent track IDs, restricted-zone highlighting, and in-zone status.

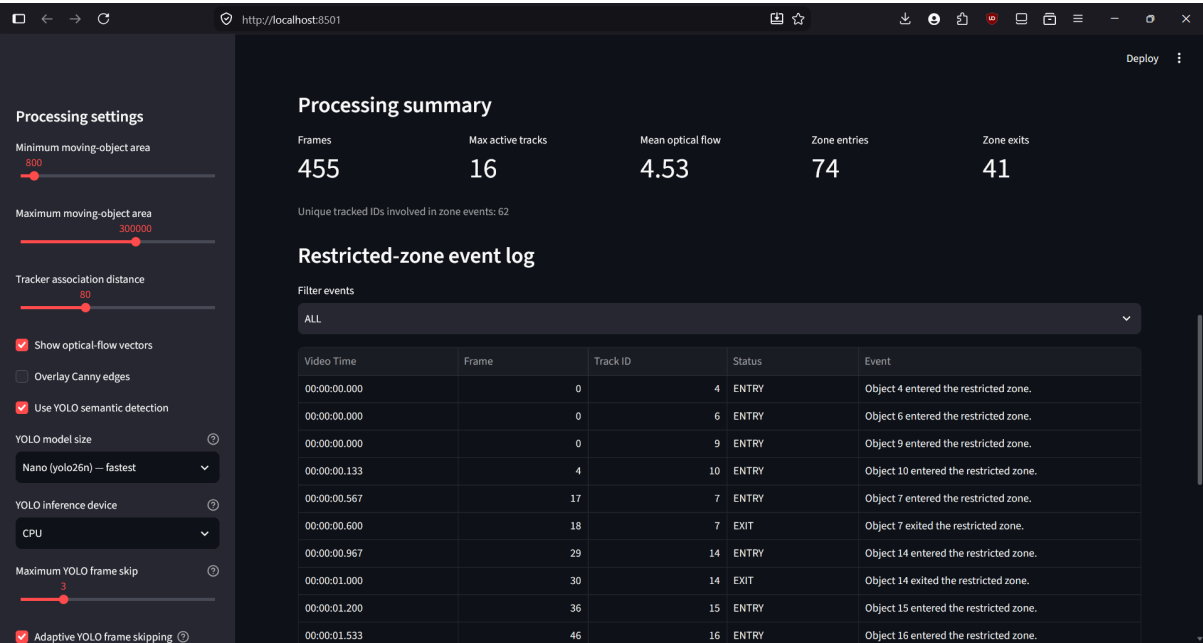


Figure 7.5: IntrudeAware processing summary and restricted-zone event dashboard.

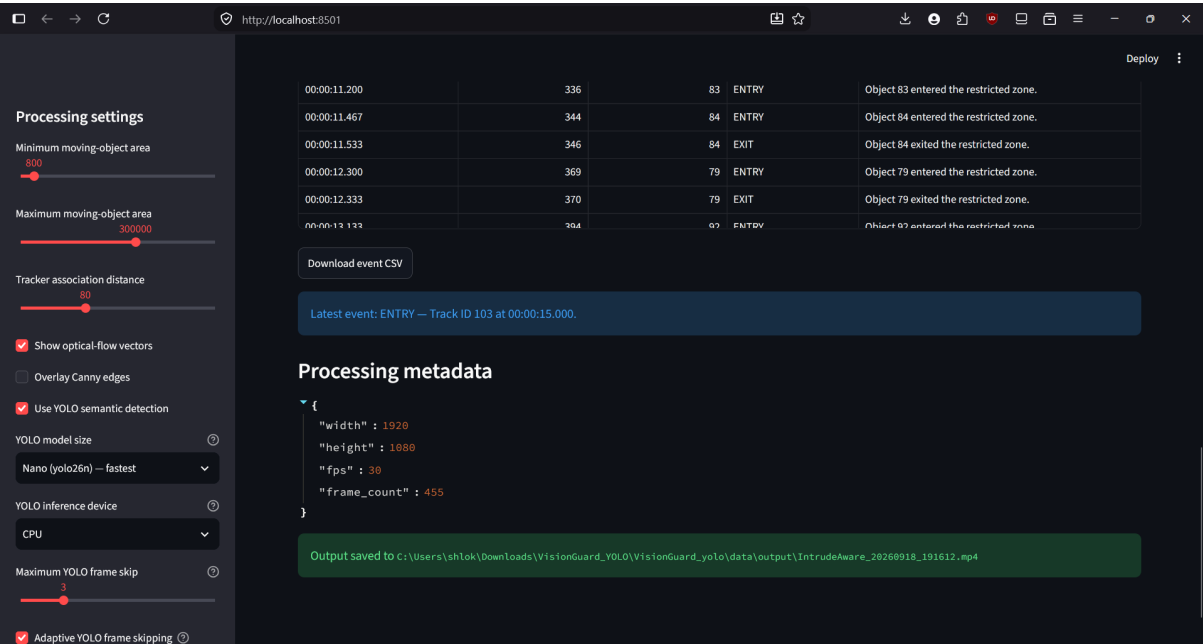


Figure 7.6: IntrudeAware event-log tail, latest event, processing metadata, and saved output path.

Table 7.1: Visible runtime settings recorded in the supplied screenshots.

Setting	Observed value
Minimum moving-object area	800
Maximum moving-object area	300000
Tracker association distance	80
Optical-flow vectors	Enabled
Canny overlay	Disabled
YOLO semantic detection	Enabled
YOLO model	Nano (yolo26n)
Inference device	CPU
Maximum YOLO frame skip	3
Adaptive YOLO frame skipping	Enabled

7.5 Observed Event-Log Evidence

A runtime event CSV was also captured from the processed video and included in the submission package as `report/evidence/intrudeaware_events.csv`. The supplied log contains 115 recorded events across 62 unique track IDs, covering video-relative timestamps from 00:00:00.000 to 00:00:15.000 and frames 0 through 450. The log contains 74 ENTRY events and 41 EXIT events. Because the log ends while some objects may still be inside the zone, the number of entries and exits need not be equal.

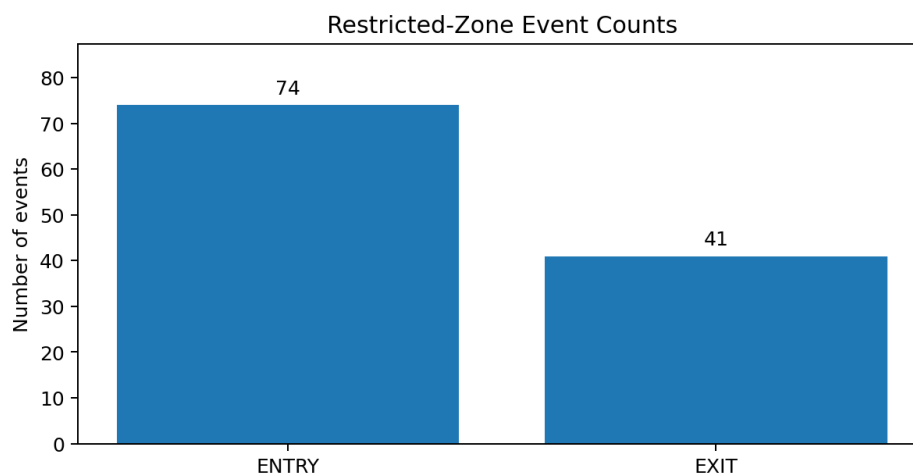


Figure 7.7: Observed ENTRY and EXIT event counts in the supplied runtime CSV log.

Table 7.2: Summary of the supplied runtime event CSV.

Metric	Observed value
Total events	115
ENTRY events	74
EXIT events	41
Unique track IDs	62
First timestamp	00:00:00.000
Last timestamp	00:00:15.000
First frame	0
Last frame	450

For traceability, the first twelve records of the supplied log are reproduced below. The complete CSV is retained in the report evidence folder.

Table 7.3: First twelve records from the supplied runtime event log.

Timestamp	Frame	Track ID	Status
00:00:00.000	0	4	ENTRY
00:00:00.000	0	6	ENTRY
00:00:00.000	0	9	ENTRY
00:00:00.133	4	10	ENTRY
00:00:00.567	17	7	ENTRY
00:00:00.600	18	7	EXIT
00:00:00.967	29	14	ENTRY
00:00:01.000	30	14	EXIT
00:00:01.200	36	15	ENTRY
00:00:01.533	46	16	ENTRY
00:00:01.767	53	17	ENTRY
00:00:01.800	54	17	EXIT

Chapter 8

Testing and Evaluation

8.1 Automated Testing

The final development snapshot contains tests for preprocessing, tracking, motion processing, semantic labeling, YOLO adapter behavior, drawing/overlay logic, and restricted-zone events. The authoring environment executed the suite with:

31 passed

within approximately 0.09 seconds. This is a software-test result for the development snapshot; it is not a video-performance benchmark.

8.2 Evaluation Methodology

The runtime screenshots and event CSV described in Chapter 9 provide qualitative and traceability evidence from an actual local run. The evaluation plan compares four scheduler configurations: every-frame YOLO, fixed frame skipping, adaptive skipping, and adaptive skipping with velocity smoothing plus refresh cooldown. All configurations should use the same input video and machine settings. Runtime, output FPS, YOLO call count, tracking statistics, flow magnitude, and zone-event quality are recorded in `evaluation/results_template.csv`.

8.3 Performance Metrics

For a labeled evaluation clip, event performance is reported using:

$$Precision = \frac{TP}{TP + FP}, \quad Recall = \frac{TP}{TP + FN},$$
$$F1 = 2 \frac{Precision \times Recall}{Precision + Recall}.$$

Entry and exit metrics should be reported separately when labels permit it. A tolerance window of plus or minus three frames is recommended for matching predicted and manually annotated transitions.

8.4 Benchmark Status

No final video benchmark values are inserted in this report because the target evaluation machine, final clip, and runtime conditions must be measured by the project author. The package includes a benchmark plan and CSV template so those measurements can be added without changing the report structure. This avoids presenting unmeasured values as project evidence.

Chapter 9

Discussion

9.1 Strengths of the Design

- The detector, tracker, event engine, and UI are modular and independently testable.
- The same track IDs are preserved across detector refreshes.
- Zone state is represented explicitly, enabling ENTRY and EXIT rather than repeated threshold alerts.
- Adaptive scheduling reduces semantic detector calls in calm scenes while prioritizing fast motion and restricted-zone activity.
- Dashboard and CSV output make system behavior inspectable instead of hiding results inside a video file.

9.2 Limitations

- The centroid tracker is a lightweight academic tracker and is not equivalent to more sophisticated multi-object tracking methods.
- YOLO predictions depend on scene quality, occlusion, lighting, model size, and confidence threshold.
- The adaptive velocity thresholds are expressed in pixels per frame and are therefore scene/camera dependent.
- The current project scope does not include camera calibration, stereo depth, 3-D reconstruction, or face recognition.
- No claim of production-grade surveillance reliability is made.

9.3 Ethical and Privacy Considerations

Video surveillance can contain personal information. For academic demonstrations, use consented or properly licensed footage. Keep stored data to the minimum needed for evaluation, avoid unnecessary identity inference, protect exported videos and event logs, and comply with institutional and applicable legal requirements. IntrudeAware is designed here as an academic video-analysis prototype rather than an autonomous decision-making system.

Chapter 10

Challenges Faced and How They Were Addressed

Challenge	Resolution
Python test imports	Added a repository-level <code>pytest.ini</code> with the project root on the Python path.
Optical-flow scalar conversion issue	Reshaped valid Lucas-Kanade point arrays before scalar extraction.
Browser video playback	Added a browser-compatible transcoding path using <code>imageio-ffmpeg</code> and H.264-compatible output.
Repeated zone events	Added explicit zone state, missing-frame grace, and re-entry cooldown.
Semantic labels	Replaced heuristic vehicle labeling with a YOLO semantic detector while keeping the existing tracker/event APIs.
YOLO compute cost	Added frame skipping, adaptive scheduling, velocity smoothing, and refresh cooldown.
Pylance overload warning	Passed an explicitly allocated output array to <code>calcOpticalFlowPyrLK</code> .

Chapter 11

Future Enhancements

1. Calibrate perspective and map pixel velocities into scene-relative measurements.
2. Evaluate stronger trackers such as ByteTrack or BoT-SORT while preserving the same event API.
3. Add line-crossing analytics and dwell-time statistics.
4. Add a labeled evaluation dataset and report precision, recall, and F1 for semantic detection and events.
5. Compare CPU, CUDA, and export formats such as ONNX/TensorRT on the same hardware.
6. Add a persistent event database if long-term logging becomes necessary.
7. Extend the system toward multi-camera views, depth estimation, or other topics from the CSE3010 syllabus in a separate research module.

Chapter 12

VITyarthi Requirement Traceability

The VITyarthi guide requires functional and non-functional requirements, architectural and workflow diagrams, UML/design artifacts, dataset/model-evaluation documentation where applicable, a GitHub repository, and a detailed report [2]. Table 12.1 maps these requirements to the project package.

Table 12.1: Traceability of submission requirements.

Requirement	Evidence in this package
Three or more major functional modules	Preprocessing, YOLO detection, tracking, motion, events, dashboard
Four or more non-functional requirements	Performance, usability, reliability, maintainability, scalability, logging
Architecture and workflow	Figures 4.1 and 4.2
UML/design diagrams	Figures 4.3, 4.4, and 4.5
Testing	<code>tests/</code> plus automated-test summary in Chapter 10
Model/dataset evaluation	<code>evaluation/benchmark_plan.md</code> and <code>evaluation/results_template.csv</code>
Repository documentation	<code>README.md</code> and <code>statement.md</code>
Final report	This PDF and its LaTeX source

Chapter 13

Conclusion

IntrudeAware integrates multiple CSE3010 computer-vision concepts into a single practical application. The final system combines preprocessing, YOLO semantic detection, persistent tracking, optical flow, restricted-zone state transitions, adaptive inference scheduling, and interactive reporting. The architecture deliberately keeps semantic detection decoupled from the tracker and event engine, enabling performance-oriented scheduling without changing the downstream behavior.

The implementation is sufficiently modular for continued development and sufficiently documented for academic evaluation. The remaining project-author task is to run the supplied evaluation protocol on the final target machine and fixed video, record the measured results in the evaluation CSV, replace the report placeholders with student information, and publish the repository to GitHub.

Appendix A

Installation and Usage

A.1 Windows PowerShell

```
cd IntrudeAware
python -m venv .venv
.\\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
python -m pytest -q
streamlit run app.py
```

A.2 Recommended first-run settings

- YOLO model: Nano (yolo26n.pt)
- Device: CPU unless CUDA is available and desired
- Confidence: 0.35
- Maximum frame skip: 3
- Adaptive frame skipping: enabled
- Velocity smoothing alpha: 0.35
- Refresh cooldown: 2 frames

Appendix B

Selected Source Files

B.1 Core dependencies

```
streamlit>=1.38,<2.0
opencv-python>=4.10,<5.0
numpy>=1.26,<3.0
pandas>=2.2,<4.0
matplotlib>=3.9,<4.0
pytest>=8.0,<9.0
imageio-ffmpeg>=0.5,<1.0
ultralytics>=8.3,<9.0
```

B.2 Project Configuration

The default YOLO configuration is represented by the following values:

```
YOLOConfig(
    model="yolo26n.pt",
    confidence=0.35,
    image_size=640,
    device="cpu",
    frame_skip=3,
    adaptive_frame_skip=True,
    velocity_smoothing_alpha=0.35,
    refresh_cooldown_frames=2,
)
```

Appendix C

References

Bibliography

- [1] VIT University / School of Computing, *CSE3010 Computer Vision Course Learning Plan*, 2020.
- [2] VITyarthi, *Build Your Own Project - General Project Instructions and Submission Guidelines*, supplied project guide.
- [3] R. Szeliski, *Computer Vision: Algorithms and Applications*, Springer, 2011.
- [4] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed., Cambridge University Press, 2004.
- [5] Ultralytics, *Ultralytics YOLO26 Documentation*, 2026. <https://docs.ultralytics.com/models/yolo26>
- [6] OpenCV, *OpenCV Documentation*, 2026. <https://docs.opencv.org/>